

Event-driven architecture

Event-driven architecture (EDA) is a software architecture paradigm concerning the production and detection of events. Event-driven architectures are evolutionary in nature and provide a high degree of fault tolerance, performance, and scalability. However, they are complex and inherently challenging to test. EDAs are good for complex and dynamic workloads. ^[1]

Overview

An *event* can be defined as "a significant change in state".^[2] For example, when a consumer purchases a car, the car's state changes from "for sale" to "sold". A car dealer's system architecture may treat this state change as an event whose occurrence can be made known to other applications within the architecture. From a formal perspective, what is produced, published, propagated, detected or consumed is a (typically asynchronous) message called the event notification, and not the event itself, which is the state change that triggered the message emission. Events do not travel, they just occur. However, the term *event* is often used metonymically to denote the notification message itself, which may lead to some confusion. This is due to Event-driven architectures often being designed atop **message-driven architectures**, where such a communication pattern requires one of the inputs to be text-only, the message, to differentiate how each communication should be handled.

This architectural pattern may be applied by the design and implementation of applications and systems that transmit events among loosely coupled software components and services. An event-driven system typically consists of event emitters (or agents), event consumers (or sinks), and event channels. Emitters have the responsibility to detect, gather, and transfer events. An Event Emitter does not know the consumers of the event, it does not even know if a consumer exists, and in case it exists, it does not know how the event is used or further processed. Sinks have the responsibility of applying a reaction as soon as an event is presented. The reaction might or might not be completely provided by the sink itself. For instance, the sink might just have the responsibility to filter, transform and forward the event to another component or it might provide a self-contained reaction to such an event. Event channels are conduits in which events are transmitted from event emitters to event consumers. The knowledge of the correct distribution of events is exclusively present within the event channel. The physical implementation of event channels can be based on traditional components such as message-oriented middleware or point-to-point communication which might require a more appropriate transactional executive framework.

Building systems around an event-driven architecture simplifies horizontal scalability in distributed computing models and makes them more resilient to failure. This is because application state can be copied across multiple parallel snapshots for high-availability.^[3] New events can be initiated anywhere, but more importantly propagate across the network of data stores updating each as they arrive. Adding extra nodes becomes trivial as well: you can simply take a copy of the application state, feed it a stream of events and run with it.^[4]

Event-driven architecture can complement service-oriented architecture (SOA) because services can be activated by triggers fired on incoming events.^{[5][6]} This paradigm is particularly useful whenever the sink does not provide any self-contained executive.

SOA 2.0 evolves the implications SOA and EDA architectures provide to a richer, more robust level by leveraging previously unknown causal relationships to form a new event pattern. This new business intelligence pattern triggers further autonomous human or automated processing that adds exponential value to the enterprise by injecting value-added information into the recognized pattern which could not have been achieved previously.

Topologies

Event driven architecture has two primary topologies. With “broker topology”, components broadcast events to the entire system without any orchestrator. This topology provides higher performance and scalability. With “mediator topology”, there is a central orchestrator which controls workflow of events. This topology provides better control and error handling capabilities. You can also use a hybrid model and combine these two topologies.^[1]

Event types

There are different types of events in EDA, and opinions on their classification may vary. According to Yan Cui, there are two key categories of events: ^[7]

Domain events

Domain events signify important occurrences within a specific business domain. These events are restricted to a bounded context and are vital for preserving business logic. Typically, domain events have lighter payloads, containing only the necessary information for processing. This is because event listeners are generally within the same service, where their requirements are more clearly understood. ^[7]

Integration events

On the other hand, integration events serve to communicate changes across different bounded contexts. They are crucial for ensuring data consistency throughout the entire system. Integration events tend to have more complex payloads with additional attributes, as the needs of potential listeners can differ significantly. This often leads to a more thorough approach to communication, resulting in overcommunication to ensure that all relevant information is effectively shared. ^[7]

Event structure

An event can be made of two parts, the event header and the event body also known as event payload. The event header might include information such as event name, time stamp for the event, and type of event. The event payload provides the details of the state change detected. An event body should not be confused with the pattern or the logic that may be applied in reaction to the occurrence of the event itself.

There are two primary methods for structuring event payloads in event-driven architectures: ^[1]

1. All necessary attributes can be included within the payload: This method enhances speed and scalability but may lead to data consistency challenges due to the presence of multiple systems of record. Additionally, it can introduce stamp coupling and bandwidth issues at scale. ^[1]
2. This method involves including only keys or IDs, allowing consumers to fetch the required data from external data sources, such as databases. While this approach is less scalable and slower due to the need for database queries, it minimizes bandwidth usage and reduces coupling issues. ^[1]

These methods represent two ends of a spectrum rather than binary choices. Architects must carefully size the event payloads to meet the specific needs of event consumers. ^[1]

Antipatterns

- The "Timeout AntiPattern," coined by Mark Richards, describes the challenges of setting timeout values in distributed systems. Short timeouts may fail legitimate requests prematurely, leading to complex workarounds, while long timeouts can result in slow error responses and poor user experiences. The Circuit Breaker Pattern can address these issues by monitoring service health through mechanisms such as heartbeats, "synthetic transactions", or real-time usage monitoring. This approach can enable faster failure detection and can improve the overall user experience in distributed architectures. ^[8]

Event evolution strategies

In event driven architectures, event evolution poses challenges, such as managing inconsistent event schemas across services and ensuring compatibility during gradual system updates. Event evolution strategies in event-driven architectures (EDA) can ensure that systems can handle changes to events without disruption. These strategies can include versioning events, such as semantic versioning or schema evolution, to maintain backward and forward compatibility. Adapters can translate events between old and new formats, ensuring consistent processing across components. These techniques can enable systems to evolve while remaining compatible and reliable in complex, distributed environments. ^[9]

Event flow layers

An event driven architecture may be built on four logical layers, starting with the sensing of an event (i.e., a significant temporal state or fact), proceeding to the creation of its technical representation in the form of an event structure and ending with a non-empty set of reactions to that event. ^[10]

Event producer

The first logical layer is the event producer, which senses a fact and represents that fact as an event message. As an example, an event producer could be an email client, an E-commerce system, a monitoring agent or some type of physical sensor.

Converting the data collected from such a diverse set of data sources to a single standardized form of data for evaluation is a significant task in the design and implementation of this first logical layer.^[10] However, considering that an event is a strongly declarative frame, any informational operations can be easily applied, thus eliminating the need for a high level of standardization.

Event channel

This is the second logical layer. An event channel is a mechanism of propagating the information collected from an event generator to the event engine^[10] or sink. This could be a TCP/IP connection, or any type of an input file (flat, XML format, e-mail, etc.). Several event channels can be opened at the same time. Usually, because the event processing engine has to process them in near real time, the event channels will be read asynchronously. The events are stored in a queue, waiting to be processed later by the event processing engine.

Event processing engine

The event processing engine is the logical layer responsible for identifying an event, and then selecting and executing the appropriate reaction. It can also trigger a number of assertions. For example, if the event that comes into the event processing engine is a product ID low in stock, this may trigger reactions such as “Order product ID” and “Notify personnel”.^[10]

Downstream event-driven activity

This is the logical layer where the consequences of the event are shown. This can be done in many different ways and forms; e.g., an email is sent to someone and an application may display some kind of warning on the screen.^[10] Depending on the level of automation provided by the sink (event processing engine) the downstream activity might not be required.

Event processing styles

There are three general styles of event processing: simple, stream, and complex. The three styles are often used together in a mature event-driven architecture.^[10]

Simple event processing

Simple event processing concerns events that are directly related to specific, measurable changes of condition. In simple event processing, a notable event happens which initiates downstream action(s). Simple event processing is commonly used to drive the real-time flow of work, thereby reducing lag time and cost.^[10]

For example, simple events can be created by a sensor detecting changes in tire pressures or ambient temperature. The car's tire incorrect pressure will generate a simple event from the sensor that will trigger a yellow light advising the driver about the state of a tire.

Event stream processing

In event stream processing (ESP), both ordinary and notable events happen. Ordinary events (orders, RFID transmissions) are screened for notability and streamed to information subscribers. Event stream processing is commonly used to drive the real-time flow of information in and around the enterprise, which enables in-time decision making.^[10]

Complex event processing

Complex event processing (CEP) allows patterns of simple and ordinary events to be considered to infer that a complex event has occurred. Complex event processing evaluates a confluence of events and then takes action. The events (notable or ordinary) may cross event types and occur over a long period of time. The event correlation may be causal, temporal, or spatial. CEP requires the employment of sophisticated event interpreters, event pattern definition and matching, and correlation techniques. CEP is commonly used to detect and respond to business anomalies, threats, and opportunities.^[10]

Online event processing

Online event processing (OLEP) uses asynchronous distributed event logs to process complex events and manage persistent data.^[11] OLEP allows reliably composing related events of a complex scenario across heterogeneous systems. It thereby enables very flexible distribution patterns with high scalability and offers strong consistency. However, it cannot guarantee upper bounds on processing time.

Extreme loose coupling and well distributed

An event-driven architecture is extremely loosely coupled and well distributed. The great distribution of this architecture exists because an event can be almost anything and exist almost anywhere. The architecture is extremely loosely coupled because the event itself doesn't know about the consequences of its cause. e.g. If we have an alarm system that records information when the front door opens, the door itself doesn't know that the alarm system will add information when the door opens, just that the door has been opened.^[10]

Semantic coupling and further research

Event-driven architectures have loose coupling within space, time and synchronization, providing a scalable infrastructure for information exchange and distributed workflows. However, event-architectures are tightly coupled, via event subscriptions and patterns, to the semantics of the underlying event schema and values. The high degree of semantic heterogeneity of events in large and open deployments such as smart cities and the sensor web makes it difficult to develop and maintain event-based systems. In order to address semantic coupling within event-based systems the use of approximate semantic matching of events is an active area of research.^[12]

Synchronous transactions

Synchronous transactions in EDA can be achieved through using request-response paradigm and it can be implemented in two ways: ^[1]

- Creating two separate queues: one for requests and the other for replies. The event producer must wait until it receives the response.
- Creating one dedicated ephemeral queue for each request.

Challenges

Event driven architecture is susceptible to the fallacies of distributed computing, a series of misconceptions that can lead to significant issues in software development and deployment. ^[1]

Finding the right balance in the number of events can be quite difficult. Generating too many detailed events can overwhelm the system, making it hard to analyze the overall event flow effectively. This challenge becomes even greater when rollbacks are required. Conversely, if events are overly consolidated, it can lead to unnecessary processing and responses from event consumers. To achieve an optimal balance, Mark Richards recommends to consider the impact of each event and whether consumers need to review the event payloads to determine their actions. For instance, in a compliance check scenario, it may be adequate to publish just two types of events: compliant and non-compliant. This method ensures that each event is only processed by the relevant consumers, reducing unnecessary workload. ^[1]

One of the challenges of using event driven architecture is error handling. One way to address this issue is to use a separate error-handler processor. So, when the event consumer experiences an error, it immediately and asynchronously sends the erroneous event to the error-handler processor and moves on. Error-handler processor tries to fix the error and sends the event back to the original channel. But if the error-handler processor fails, then it can send the erroneous event to an administrator for further inspection. Note that if you use an error-handler processor, erroneous events will be processed out of sequence when they are resubmitted. ^[1]

Another challenge of using event driven architecture is data loss. If any of the components crashes before successfully processing and handing over the event to its next component, then the event is dropped and never makes it into the final destination. To minimize the chance of data loss, you can persist in-transit events and remove / dequeue the events only when the next component has acknowledged the receipt of the event. These features are usually known as "client acknowledge mode" and "last participant support". ^[1]

See also

- Event-driven programming
- Process Driven Messaging Service
- Service-oriented architecture
- Event-driven SOA
- Space-based architecture
- Complex event processing
- Event stream processing
- Event Processing Technical Society
- Staged event-driven architecture (SEDA)
- Reactor pattern

- Autonomous peripheral operation

Articles

- Article defining the differences between EDA and SOA: *How EDA extends SOA and why it is important* (<https://soa-eda.blogspot.com/2006/11/how-eda-extends-soa-and-why-it-is.html>) by Jack van Hoof.
- Real-world example of business events flowing in an SOA: *SOA, EDA, and CEP - a winning combo* (<http://www.udidahan.com/2008/11/01/soa-eda-and-cep-a-winning-combo/>) by Udi Dahan.
- Article describing the concept of event data: *Analytics for hackers, how to think about event data* (<https://web.archive.org/web/20160405073553/https://keen.io/blog/53958349217/analytics-for-hackers-how-to-think-about-event-data>) by Michelle Wetzler. (Web archive)

References

1. Richards, Mark. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media. ISBN 978-1492043454.
2. K. Mani Chandy *Event-driven Applications: Costs, Benefits and Design Approaches*, California Institute of Technology, 2006
3. Martin Fowler, *Event Sourcing* (<https://martinfowler.com/eaDev/EventSourcing.html>), December, 2005
4. Martin Fowler, *Parallel Model* (<https://martinfowler.com/eaDev/ParallelModel.html>), December, 2005
5. Hanson, Jeff (January 31, 2005). "Event-driven services in SOA" (<https://www.infoworld.com/article/2072262/event-driven-services-in-soa.html>). *JavaWorld*. Retrieved 2020-07-21.
6. Sliwa, Carol (May 12, 2003). "Event-driven architecture poised for wide adoption" (<https://www.computerworld.com/article/2570503/event-driven-architecture-poised-for-wide-adoption.html>). *Computerworld*. Retrieved 2020-07-21.
7. Cui, Yan. *Serverless Architectures on AWS*. Manning. ISBN 978-1617295423.
8. Richards, Mark. *Microservices AntiPatterns and Pitfalls*. O'Reilly.
9. *Designing Event-Driven Systems*. O'Reilly Media. ISBN 9781492038245.
10. Brenda M. Michelson, *Event-Driven Architecture Overview*, Patricia Seybold Group, February 2, 2006
11. "Online Event Processing - ACM Queue" (<https://queue.acm.org/detail.cfm?id=3321612>). *queue.acm.org*. Retrieved 2019-05-30.
12. Hasan, Souleiman, Sean O'Riain, and Edward Curry. 2012. "Approximate Semantic Matching of Heterogeneous Events." (http://www.edwardcurry.org/publications/Hasan_DEBS_2012.pdf) In 6th ACM International Conference on Distributed Event-Based Systems (DEBS 2012), 252–263. Berlin, Germany: ACM. "DOI" (<https://dx.doi.org/10.1145/2335484.2335512>).

External links

- *Event-Driven Applications: Costs, Benefits and Design Approaches* (<http://infospheres.caltech.edu/sites/default/files/Event-Driven%20Applications%20-%20Costs,%20Benefits%20and%20Design%20Approaches.pdf>) Archived (<https://web.archive.org/web/20131023055205/http://infospheres.caltech.edu/sites/default/files/Event-Driven%20Applications%20-%20Cost>

[s,%20Benefits%20and%20Design%20Approaches.pdf](#)) 2013-10-23 at the [Wayback Machine](#)

- [5th Anniversary Edition: Event-Driven Architecture Overview](http://www.elementallinks.com/2011/02/06/5th-anniversary-edition-event-driven-architecture-overview/), Brenda M. Michelson (<http://www.elementallinks.com/2011/02/06/5th-anniversary-edition-event-driven-architecture-overview/>)
- [Complex Event Processing and Service Oriented Architecture](http://news.tmcnet.com/news/2006/08/18/1816129.htm) (<http://news.tmcnet.com/news/2006/08/18/1816129.htm>)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Event-driven_architecture&oldid=1362530550"